DAY 10 | HANDS-ON ACTIVITIES

Async JavaScript and Node.js: Problem Statements

Seven problems. Run JavaScript with Node.js (in Colab: !node file.js). Test the API with Postman or a client script.

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

How to work: read the problem, write the steps in plain English first, then write the code. Test with the sample run, then try your own inputs, including wrong ones.

Problem 1: Predict the Order

Level: Warm-up | **Concepts:** event loop, setTimeout

Write a script with three console.log calls and two setTimeout calls (0 ms and 100 ms) mixed in. Predict the output on paper, then run it and compare.

Sample run / expected result

```
A
C
B (timeout 0)
D (timeout 100)
```

Hints

- Synchronous code always finishes before any timer callback.

Problem 2: Promise Kitchen

Level: Easy | **Concepts:** new Promise, then, catch

Write cook(dish, ms) that resolves after ms milliseconds, or rejects if dish is "pizza" (out of cheese). Chain three dishes with then and handle the error with catch.

Sample run / expected result

```
rice ready
curry ready
Error: out of cheese for pizza
```

Hints

- return new Promise((resolve, reject) => setTimeout(...))

Problem 3: Sequential vs Parallel

Level: Medium | **Concepts:** async/await, Promise.all, timing

Cook three dishes of 500 ms each sequentially with await, then in parallel with Promise.all. Print how long each approach took.

Sample run / expected result

```
Sequential: 1503 ms
Parallel: 502 ms
```

Hints

- const start = Date.now(); ... Date.now() - start

Problem 4: Express Tasks API

Level: Medium | **Concepts:** Express routes, express.json, status codes

Build GET /tasks, GET /tasks/:id, POST /tasks (400 without title, 201 on success), PUT /tasks/:id and DELETE /tasks/:id (204). Return 404 for unknown ids.

Sample run / expected result

```
POST /tasks {} -> 400 {"error": "title is required"}
DELETE /tasks/1 -> 204
```

Hints

- Number(req.params.id) converts the id.
- res.status(204).end()

Problem 5: Logging Middleware

Level: Easy | **Concepts:** app.use, next()

Add middleware that logs method, path, status and time taken for every request, and a catch-all 404 handler for unknown routes.

Sample run / expected result

```
POST /tasks 201 3ms
GET /nope 404 1ms
```

Hints

- res.on("finish", ...) runs after the response is sent.

Problem 6: Postman Collection

Level: Medium | **Concepts:** Postman, environments, tests

Create a collection with one request per endpoint using {{baseUrl}}. Add at least one test per request that checks the status code. Run the whole collection.

Sample run / expected result

```
Collection run: 6 requests, 6 tests passed
```

Hints

- Import the collection generated in the notebook as a starting point.

Problem 7: Async Client

Level: Stretch | **Concepts:** fetch, async/await, error handling

Write client.js that creates three tasks in parallel, marks one done, deletes another, then prints the final list. Handle a non-2xx response by throwing an error with the status.

Sample run / expected result

```
Created 3 tasks
Final: [{id: 1, done: true}, {id: 3, done: false}]
```

Hints

- if (!res.ok) throw new Error(`HTTP ${res.status}`)

Ganbatte (頑張って): do your best